

DAVE: Deriving Automatically Verilog from English

Hammond Pearce
hammond.pearce@nyu.edu
New York University
Brooklyn, USA

Benjamin Tan
benjamin.tan@nyu.edu
New York University
Brooklyn, USA

Ramesh Karri
rkarri@nyu.edu
New York University
Brooklyn, USA

ABSTRACT

Specifications for digital systems are provided in natural language, and engineers undertake significant efforts to translate these into the programming languages understood by compilers for digital systems. Automating this process allows designers to work with the language in which they are most comfortable — the original natural language — and focus instead on other downstream design challenges. We explore the use of state-of-the-art machine learning (ML) to automatically derive Verilog snippets from English via fine-tuning GPT-2, a natural language ML system. We describe our approach for producing a suitable dataset of novice-level digital design tasks and provide a detailed exploration of GPT-2, finding encouraging translation performance across our task sets (94.8 % correct), with the ability to handle both simple and abstract design tasks.

CCS CONCEPTS

• **Computing methodologies** → **Machine translation**; • **Hardware** → **Hardware description languages and compilation**.

ACM Reference Format:

Hammond Pearce, Benjamin Tan, and Ramesh Karri. 2020. DAVE: Deriving Automatically Verilog from English. In *2020 ACM/IEEE Workshop on Machine Learning for CAD (MLCAD '20)*, November 16–20, 2020, Virtual Event, Iceland. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3380446.3430634>

1 INTRODUCTION

In pursuit of simplifying and acceleration digital design, a machine-driven design flow with “no humans in the loop” is a long-term goal of projects such as OpenROAD¹. Typically, the starting point is human-prepared hardware specifications in a Hardware Description Language (HDL) such as Verilog. However, manually producing HDL to match a given specification (e.g. in Fig. 1) requires significant domain knowledge and is challenging to write error-free. As such, there is an opportunity for automatic translation to increase productivity and reduce the burdens on human designers. Given successful adoption of Machine Learning (ML) throughout the Integrated Circuit (IC) Computer-Aided Design (CAD) flow (e.g. [7, 14, 20]), we are motivated to investigate if state-of-the-art ML can help in even earlier design stages.

¹<https://theopenroadproject.org/>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MLCAD '20, November 16–20, 2020, Virtual Event, Iceland
© 2020 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-7519-1/20/11...\$15.00
<https://doi.org/10.1145/3380446.3430634>

ML has recently made great strides in Natural Language Processing (NLP). Advances in Deep Learning (DL) have included new architectures such as LSTMs [15], RNNs [8], and Transformers [18]. These architectures have led to models such as BERT [3] and GPT-2 [12] which demonstrate capability in language modelling, language translation (e.g., English to French), reading comprehension/understanding (e.g., answering questions from the CoQA [13] dataset), and information storage/retrieval. In fact, GPT-2 made headlines [6] for initially being “too dangerous” to release given the “quality” of its text generation. *Can we harness this power to produce hardware from task descriptions (like in Fig. 1)?*

Towards the goal of fully automated design from natural language, we investigate the adaptation of a pre-trained natural language model to perform English to Verilog “translation”. Using transfer learning [10], we fine-tune the recently presented GPT-2 for this task by training it on a custom dataset of Task/Result pairs, as in Fig. 1. The tasks are somewhat akin to novice-level “textbook” problems (i.e., similar to those found in a classic textbook [17]). We validate our approach by presenting a set of “unseen” tasks to translate and measure the quality of output. Our contributions are:

- DAVE, a pre-trained GPT-2 model that can translate natural language into Verilog implementation.
- A method to automatically generate a large quantity of English specification, Verilog pairs for fine-tuning DAVE.
- Exploration and evaluation of fine-tuning DAVE.
- Rating DAVE in translating complex *descriptive* tasks besides those presented in simple *prescriptive* forms.

The rest of the paper is as follows. Section 2 provides background and discuss related work. Section 3 describes our experimental approach and dataset preparation. Section 4 presents the results of our experimental investigation. Section 5 concludes.

2 BACKGROUND AND RELATED WORK

ML-CAD. ML techniques, including DL have shown promising results across numerous applications, including across the CAD domain. Recent work spans the design flow, from early-stage hardware cost estimations [14], through logic synthesis [20], and physical

TASK: Write sequential code for a call button (e.g., in an airplane or hospital). If the call button ‘b’ is pressed (= 1) then the call light ‘l’ should turn on (= 1). The output call light ‘l’ should turn off (= 0) when the synchronous cancel button ‘r’ is pressed (= 1).

RESULT:

```
// assume clock clk
reg l;
always @(posedge clk) begin
  if (r) begin
    l <= 0;
  end else if (b) begin
    l <= 1;
  end
end
```

Figure 1: Example “Task” and Corresponding Verilog

design [7]. We explore the use of transfer learning [10] to teach a DL-based model to produce Verilog by framing it as a machine translation problem. Transfer learning provides the ability to learn new tasks without large quantities of labelled data in a target domain.

GPT-2. We use GPT-2 [12] as our starting point, given its state-of-the-art performance in zero-shot task settings. GPT-2 is based on the decoder part of the Transformer, a neural network encoder-decoder architecture with a self-attention mechanism [18]. At the core of the GPT-2 approach is language modelling, which can be framed as an unsupervised distribution estimation from some set of examples $(x_1, x_2, \dots, x_n)$, where each example is composed of variable length sequences of symbols $(s_1, s_2, \dots, s_n)$ [12]. This statistical model of language is thus the joint probability distribution of the symbols in the language (as the product of the conditional probabilities for each symbol given the preceding sequence [1]). Put simply, the model learns to answer the following: *given some sequence of symbols, what is the most likely next symbol in the sequence?*

Different tasks can be specified in a language itself, e.g., [*“translate to french”, “english text”, “french text”*] [12]. Radford *et al.* speculate that a model with sufficiently large capacity can learn to perform tasks demonstrated in natural language without explicit supervision. In other words, given a general system which produces $p(\text{output}|\text{input})$, a condition can be introduced to model some task $p(\text{output}|\text{input}, \text{task})$. By training GPT-2 on a large, unlabelled dataset (~ 8 million webpages), Radford *et al.* demonstrated the trained model could perform well on numerous tasks without fine-tuning. The trained model then provides a good starting point for performance in specific tasks following fine-tuning [11]. Fundamentally, GPT-2’s pre-trained, implicit capability to process natural language can be directed towards specific tasks. We attempt to harness this capability by fine-tuning GPT-2 for translating natural language descriptions to Verilog.

Natural Language $\rightarrow$ Code. The challenges in translating specifications into computer code has driven research in natural language programming [9]. Recent work has shown that there is a finite limit to the number of unique ways one can express certain programming structures (e.g. *for*-loops) in natural language, and as such it is possible to extract this information and transform it into its corresponding computer code [9]. Other related works use NLP techniques, including rule-based processing, for formal system modeling [4], generating hardware assertions [5], and for enhancing documentation by automatically extracting software development tasks and associating them with the relevant paragraphs [16]. While showing promising results, there are limitations on how flexible the natural language descriptions can be with respect to structure. Earlier work involves designing separate components to perform specific tasks such as identifying “steps”, “loops”, and “comments” from natural text [9]. To our knowledge, DL techniques to generate HDL from natural language have not been explored.

3 FINE-TUNING GPT-2 FOR VERILOG

3.1 Problem definition

In this work, we focus on an early-stage CAD problem: interpreting a high-level, informal description of functionality and producing the corresponding concrete specification. For small designs, designers can craft an RTL specification directly after identifying

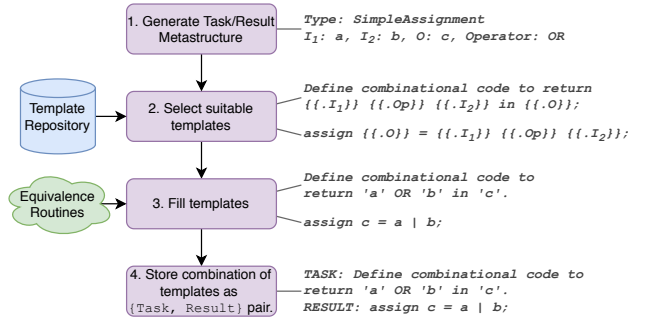


Figure 2: The Task/Result Generation Process

the necessary inputs, outputs, and the relationships between them from a short description of a task. While previous works use algorithmic approaches such as parse-tree generation and sub-tree matching [21] to identify the salient elements of the natural language description for populating templates, we re-cast the problem holistically as *translation*. As we describe next, we prepare examples of task descriptions with varying descriptiveness, and examine GPT-2’s ability to produce Verilog after transfer learning [10].

3.2 Dataset Preparation

In this work, we fine-tune GPT-2 to produce DAVE, aiming for the ability to translate natural language (i.e., English) into Verilog. GPT-2 is designed to process contiguous text sequences, so we adopt the approach proposed in [11], to represent the English–Verilog translation task as an ordered sequence in the format ‘TASK: <English Text> RESULT: <Verilog Code>’.

Open-source Verilog code can be found online, but is unstructured, with varying quality and complexity. For this initial study, we design a custom dataset generation tool inspired by the sort of template-based, random auto-marking Q&A systems used in teaching settings (e.g., the OASIS Question Engine²). Rather than produce thousands of Task/Result pairs manually, we prepare several natural language *templates* which encapsulate different task scenarios. An example generation process is shown in Fig. 2.

In step (1) our tool generates a Task/Result *metastructure*, a descriptor for the type of task (e.g., an assignment) and relevant information for that task (e.g., variable names, operators). Possible metastructure tasks include combinational signal *assignments*, *registers*, *sequence generators*, or a multi-set of these. Then, in step (2), the tool randomly chooses a suitable template for the task that encapsulates all information in English and Verilog. In step (3), the tool “fills in” these templates, translating arguments where necessary (e.g. OR operator is ‘or’ in English and ‘|’ in Verilog). Finally, in step (4), the tool saves the generated Task/Result pair.

Structurally, we organise our templates into the different task classes they describe—(combinational) assignments, registers, and sequence generators. We then categorise them further as either *prescriptive* or *descriptive*. **Prescriptive templates** are like the example presented in Fig. 2. We conjecture that these should be trivial to translate—simple substitutions and word-reordering is all that is required to convert from the English to Verilog. **Descriptive templates**, meanwhile, are more like the example presented in

²<https://www.oasisqe.com/>

Table 1: Template-based Dataset Information. (pX → prescriptive; dX → descriptive; X is the task type)

Task	# for Training	# Non-Training	Samples / Template	Example of Task in English	Model Verilog	
Assignment (a)	pa	17	2	2000	<i>Given inputs 'a' and 'b', take the nor of these and return the result in 'c'.</i>	<code>assign c = !(a b);</code>
	da	3	1	4000	<i>A house has three active-low alarm detector triggered sensors 'a', 'b', 'c'. Write combinatorial logic for a active-high light 'l' which activates when any of the detectors are triggered.</i>	<code>assign l = !(a & b & c);</code>
Register (r)	pr	9	2	3000	<i>Define a 4-bit register 'q' with input 'a' nand 'b', enable 'e' defined as 'b' xnor 'r', an asynchronous reset 'r', and a clock 'c'.</i>	<code>assign e = b ^ r; reg q; always @(posedge c or posedge r) begin if(r) begin q <= 0; end else if(e) begin q <=!(a & b); end end;</code>
	dr	3	1	4000	<i>Design the code for an alarm system. When the panic mode 'm' is selected (= 0) the alarm system 'a' should activate (= 1) and should only deactivate (= 0) when the active-low synchronous cancel button 'c' is selected (= 1).</i>	<code>//assume clock clk reg a; always @(posedge clk) begin if(c) begin a <= 0; end else if(!m) begin a <= 1; end end</code>
Sequence Generator (g)	pg	4	2	4000	<i>Define sequential code which will produce the repeating sequence [0, 1, 0] on output 'u'. It should advance on clock 'c' whenever enable 'e' is present, and a synchronous reset 'r' should reset the sequence back to the first element.</i>	<code>enum {s0, s1, s2} state; reg u; always @(posedge c) begin if(s) begin state <= s0; u <= b0; end else begin unique case (state) s0: if(e) begin state <= s1; u <= b0; end s1: if(e) begin state <= s2; u <= b1; end s2: if(e) begin state <= s0; u <= b0; end endcase end</code>
Multi-task (M-T)	-	N/A	N/A	5250	<i>Write a 6-bit register 'ar' with input defined as 'gv' modulo 'lj', enable 'q', synchronous reset 'r' defined as 'yxo' greater than or equal to 'm', and clock 'p'. A vault door has three active-low secret switch pressed sensors 'et', 'lz', 'l'. Write combinatorial logic for a active-high lock 's' which opens when all of the switches are pressed. Write a 6-bit register 'w' with input 'se' and 'md', enable 'mmx', synchronous reset 'nc' defined as 'tfs' greater than 'w', and clock 'xx'.</i>	<code>assign r = yxo >= m; reg [5:0] ar; always @(posedge p) begin if(r) begin ar <= 0; end else if(q) begin ar <= gv % lj; end end assign s = !(et lz l); assign nc = tfs > w; reg [5:0] w; always @(posedge xx) begin if(nc) begin w <= 0; end else if(mmx) begin w <= se & md; end end</code>

Fig. 1. They are more complex to translate, and a human designer would implicitly perform intermediate steps—such as understanding that a given input is being used as an enable signal or as a reset. **Multi-task** templates are random concatenations of two to four assignment/register templates. Table 1 provides additional examples of the different task types generated from the various templates.

While at first glance this template-based approach for dataset generation might appear to restrict DAVE’s ability to generalize over English descriptions, this dataset is only used for *fine-tuning* the language model. As GPT-2 is pre-trained over the large WebText dataset [12], we theorize that DAVE should retain at some ability to process natural language features such as synonyms and different word/clause orders. To validate this hypothesis, we hold-out a subset of templates for use during testing and evaluation. Table 1 has information about the final dataset, including the number of “Trained” and “Non-Trained” (held-out) templates for all task types.

In our evaluation, we initially query DAVE with new task instances based on Trained templates to observe its baseline ability to perform “familiar” tasks (i.e., produce Verilog from English descriptions that are similar to the training data). To study generalizability of the approach, we query DAVE with new task instances based on

Non-Trained templates, i.e., such Task/Result pairs are presented to the language model during validation.

While the number of templates might appear low in certain cases (e.g., # of Descriptive vs. Prescriptive assignments), the task instances of the given templates vary significantly from each other due to the addition or omission of optional clauses in the natural text during data generation. A template that describes a register design task may have a clause describing a reset signal, and if the template is used for a metastructure with no reset signal, that entire clause is omitted. As such a given template identifier refers only to the overall sentence structure used in a Task, the unique pattern of compulsory words within that template, such as introductory remarks (e.g. “Describe combinatorial logic to...”), and individual words used within that template (e.g. conjunctions, prepositions). Descriptive templates have randomly generated *settings* such as “an attendant call button”. These are generated from the cascaded sub-templates, increasing the entropy of each individual Task/Result pair. Register and Sequence Generator templates are allowed to recursively define the basic template (prescriptive assignments). A register might define a signal (e.g. an enable) as a function (e.g. ‘a’ nand ‘b’) rather than as a pre-set input (e.g. ‘c’).

Multi-tasks combine other types of tasks and are difficult to categorise. We randomly generate 5,250 multi-task samples, of which 5000 are used for fine-tuning. We discuss details in Section 4.4.

3.3 Experimental Platform

After we generate a suitable corpus of Task/Result pairs according to the method described in Section 3.2, we fine-tune the 345 million parameter GPT-2 model on a high-performance computing node with 2 Intel Xeon E5-2698 v4 @ 2.20GHz cores, 20 GB of RAM, and an NVIDIA V100 32 GB graphics card over all categories of Task/Result pairs simultaneously (i.e. the same trained model is used to decode each type of Task). Our fine-tuning script is modified from [19]. We use the Python programming environment, with *pytorch* version 1.5.0, *tensorflow* version 2.2, and *aitextgen* version 0.2.3. Underlying these we use *cuda* version 10.1 and *cuda* version 7.6.

To fine-tune GPT-2, we leave the hyper-parameters at their suggested defaults (*learning rate* 1e-4, *weight decay* 0.05, *adam epsilon* 1e-8) and perform fine-tuning for 7500 steps. The training data covers a random sample of 95% of the generated samples of each Trained template category, with 5% held back for evaluating the model. To evaluate model “goodness”, we use the same computing resources as for training and use default GPT-2 output generation parameters (*temperature* 0.7, *top_p* 0.9, and *top_k* 0/disabled).

4 EXPERIMENTAL INVESTIGATION

4.1 Overview

The purpose of this work is to explore the potential for general-purpose language models in translating system specifications provided in English to their hardware implementations in the Verilog HDL. As such we are interested in measuring the quality of the generated Verilog. This raises an obvious question—how should one define “quality”? In this work we are interested in a language model which can perform design tasks of a similar difficulty to those posed in a textbook [17].

However, there are no automated systems to quantify how well a specification has been implemented in its corresponding Verilog if it is “almost” correct. Formal equivalence check is an option, but requires that the design is at least syntactically compliant. This presents a challenge as we wish to quantify the quality of DAVE’s Verilog generation. However, given that we generate Task/Result pairs with a template engine, we have a baseline ‘canonical’ response that we can compare DAVE’s output against. This allows us to introduce the equivalence between the two generators as a measure of quality, discussed in subsection 4.1.1. Where DAVE’s output is not equivalent, we manually examine the result qualitatively.

An important part of our evaluation is to examine DAVE’s performance over unfamiliar texts. Otherwise, it could be argued that the language model has simply learned a kind of pattern recognition over the Task/Result pairs, and is just using string relocation techniques to score highly during validation. If this notion were applied to a student, we might say that they had learned to produce Verilog by rote, rather than through understanding.

This examination is provided through the Non-Trained Templates. Recall that these are unfamiliar to DAVE, i.e., they were not seen during fine-tuning, and DAVE has had no opportunity to learn/memorize their syntax and structure. We seek insight from

DAVE’s performance over these tasks as evidence that the GPT-2 language model offers promise for our intended translation purpose.

4.1.1 A measure of equality. There are numerous ways to implement a given specification in any programming language. Take the example from Fig. 2: while it provides the correct answer as `assign c = a | b;`, it could be equivalently specified as `assign c = b | a;`. This becomes even more of an issue when implementing larger and more complex and descriptive specifications.

While there are ways of quantifying identical code (e.g., comparing abstract syntax trees), we opt, for a simpler comparison of DAVE’s outputs against the template tool using a sequence equivalence metric. This is because the generated Verilog code should be relatively short and simple. More precisely, we define *correctness* of the generated text as its distance to the template-provided “correct” answer (excluding white-space characters from both) as measured by their Ratcliff-Obershelp similarity [2]. This means that if DAVE returns `assign c = a | b;` as the correct answer to the prompt in Fig. 2, it scores 1.00—i.e., the result is fully correct. However, despite being functionally equivalent, a result of `assign c = b | a;` scores only 0.833.

While this metric is simple, manual inspection of the results that did not have the expected score of 100, revealed no examples where DAVE had performed small but functionally equivalent changes (e.g., inverting the order of variables compared to their order in the specification). That the output has a deterministic ordering to the variables is not a surprising result, as the template engine that DAVE is fine-tuned from has a deterministic order to the Verilog code that it produces. We provide insights from our investigation in three parts: DAVE’s performance on prescriptive (Section 4.2), descriptive (Section 4.3), and multi tasks (Section 4.4).

4.2 Translation of Prescriptive Specifications

DAVE’s performance on prescriptive tasks is presented in Table 2, with Non-Trained templates highlighted in **bold**. Each row contains information on the number of template samples used for fine-tuning, the number of template samples used for validation, the number DAVE returned correctly, and (where applicable) the average Ratcliff-Obershelp (R-O) similarity of returned incorrect answers compared to the correct answer.

With regards to assignments, DAVE performs well on tasks based on Trained (e.g., *pa00*³) templates, getting 99.7% of all samples correct across this validation category. It performs slightly worse on tasks drawn from Non-Trained templates (e.g., *pa18*⁴), scoring 96.5% correct. DAVE scores well on Trained register templates (e.g., *pr00*⁵) (99.2% correct). Likewise DAVE performed well with the Non-Trained Templates in this category (e.g. *pr11*⁶), with 98.7% correct. While DAVE did well in Trained Sequence Generators (e.g. *pg01*⁷) with 99.5% correct across the samples, it performed poorly

³*pa00* example: “Put the result of ‘a’ nand ‘b’ in ‘c’.”

⁴*pa18*: “Assign into output ‘c’ the result of ‘a’ xor ‘b’.”

⁵*pr00*: “Define a 8-bit register ‘a’ with input ‘a’ defined as ‘b’ and ‘c’, enable ‘e’, and clock ‘c’.”

⁶*pr11*: Given input ‘a’, enable ‘e’ defined as ‘d’ nxor ‘f’, an asynchronous reset ‘r’ (being ‘x’ or ‘y’) make a 7-bit register ‘q’.

⁷*pg01*: “Define sequential code which will produce the repeating sequence [00, 10, 10] on the 2-bit output ‘q’. It should advance on each tick of a clock ‘c’ whenever enable defined as ‘a’ nxor ‘b’ is present.”

Table 2: Testing DAVE on Prescriptive Tasks

Type	Template Name	# Trained	# Validated	# Correct	Avg. Error R-O
Assignment	pa00	1900	100	99	0.947
	pa01	1900	100	100	–
	pa02	1900	100	100	–
	pa03	1900	100	100	–
	pa04	1900	100	100	–
	pa05	1900	100	100	–
	pa06	1900	100	97	0.951
	pa07	1900	100	100	–
	pa08	1900	100	100	–
	pa09	1900	100	100	–
	pa10	1900	100	100	–
	pa11	1900	100	100	–
	pa12	1900	100	100	–
	pa13	1900	100	100	–
	pa14	1900	100	99	0.947
	pa15	1900	100	100	–
	pa16	1900	100	100	–
	pa17	0	100	95	0.956
	pa18	0	100	98	0.898
Register	pr00	2850	150	148	0.981
	pr01	2850	150	149	0.993
	pr02	2850	150	149	0.973
	pr03	2850	150	150	–
	pr04	2850	150	148	0.990
	pr05	2850	150	147	0.982
	pr06	2850	100	148	0.993
	pr07	2850	150	149	0.983
	pr08	2850	150	150	–
	pr09	2850	150	150	–
	pr10	0	150	149	0.960
	pr11	0	150	147	0.965
Seq. Generator	pg01	3800	200	200	–
	pg02	3800	200	199	0.996
	pg03	3800	200	200	–
	pg04	3800	200	197	0.984
	pg05	0	200	200	–
	pg06	0	200	143	0.889

with the Non-Trained template *pg06*⁸, bringing the overall percentage correct for Non-Trained Templates down to 85.6 %.

Discussion. One would expect DAVE to perform well on tasks produced from Trained templates, given that these most resemble the training data. This held true for all three major categories. One might also expect that DAVE would perform worse on task prompts generated from Non-Trained templates in comparison to prompts generated from the Trained templates. Our hypothesis is that the GPT-2 pre-training should allow DAVE to generalise and produce the correct Verilog even in unseen tasks.

This holds for Assignments and Registers, but did not entirely hold with the Non-Trained Sequence Generator templates, specifically with *pg06*. Closer investigation of this template revealed that almost all of DAVE’s errors (>95 %) stem from mis-classification of enable and reset signals. This was unexpected as DAVE did not have this issue over tasks based on any other Sequence Generator template. One theory is that the issue may stem from the difference between *pg06* and the other templates—perhaps it is *too* unique. To evaluate this, we compared the the R-O similarity of templates *pg05* (which scored 100 %) and *pg06* with the Trained *pg* templates. We found that *pg05* was closest to *pg01* (similarity 0.820), whereas *pg06*

⁸*pg06*: “Produce a design that generates a 3-bit output ‘uy’ with the sequence: [110, 100, 101, 100]. The output changes with each rising edge of a clock if the enable signal ‘a’ less than ‘b’ is asserted. Whenever an asynchronous reset ‘r’ is asserted, the design should output the first element of the sequence.”

Table 3: Testing DAVE on Descriptive and Multi- Tasks

Type	Template Name	# Trained	# Validated	# Correct	Avg. Error R-O
Assign.	da00	3800	200	200	–
	da01	3800	200	199	0.952
	da02	3800	200	196	0.956
	da03	0	200	200	–
Register	dr00	3800	200	200	–
	dr01	3800	200	195	0.985
	dr02	3800	200	199	0.992
	dr03	3800	200	198	0.988
	dr04	0	200	196	0.987
M-T	Trained	5000	250	130	0.907
	Non-Trained	0	250	103	0.817

was closest to *pg03* (similarity 0.777). These numbers are similar enough that we would have expected *pg06* to score better. Further formal analysis is an avenue for our future work. It is likely that providing a greater variety of Sequence Generator templates during training would help DAVE produce more accurate results.

4.3 Translation of Descriptive Specifications

Table 3 presents DAVE’s performance over Descriptive Tasks. While this category has fewer templates, each template has more opportunities for entropy due to the presence of optional clauses and implicit intermediate signals. We also design these templates to be more “difficult”—they invoke requirements such as ‘active-high’ and ‘active-low’ qualifiers to their variables, terms that DAVE needs to recognise and accommodate in the generated Verilog.

Somewhat surprisingly, DAVE performs better on Descriptive Tasks than on the Prescriptive Tasks, with 99.2 % correct Assignments and 99.0 % Registers over the Trained Templates. For the Non-Trained templates, the Assignments scored 100 % correct and Registers scored 98 %. To check that this high score was not due to the Non-Trained templates *da03* and *dr04* being structurally similar to the Trained templates, we compare R-O similarities. *da03* is most similar to *da01*, with a score of 0.686. *dr04* is most similar to *dr02*, with a score of 0.703. While these values might seem high, consider the Sequence Generator template *pg06*, which scored 0.777 yet DAVE gave the correct answer only 71.5 % of the time.

Discussion. On a number of occasions, we were particularly impressed that DAVE was able to derive the Boolean combinations for certain operations. Take this example from *da00*: “A car has four active-low door open sensors ‘a’, ‘b’, ‘c’, ‘d’. Write combinatorial logic for a active-low light ‘l’ which illuminates when any of the doors are open.” From that prompt, DAVE is able to correctly generate the output `assign l = a & b & c & d;`, i.e., it appears to associate ‘any’ and ‘doors’, as well as understand the relationship between ‘any’ and the two ‘active-low’ qualifiers. Another example of DAVE “understanding” keywords is the generated Verilog for *dr00*, which we present in Fig. 1. DAVE can correctly implement both synchronous and asynchronous resets, as well as infer clocks for memory elements when no clocks are explicitly specified.

4.4 Translation of Multiple Tasks

For insight into how DAVE can handle the processing of multiple tasks simultaneously we also provided a multi-task metastructure consisting of 2-4 registers and assignments in a single Task prompt. These are presented in Table 3 under M-T. We divide Multi-tasks

into two broad categories—those made purely from Trained templates (of which 5000 were presented during the fine-tuning process), and those made only from Non-Trained templates. Multi-tasks performed worse than the individual templates (Trained correct 52 % of the time, and Non-Trained 41.2 %). Upon manual inspection, DAVE was generating the correct Verilog structures and syntax in the outputs, usually only getting variable names/operators incorrect. This is reflected in the Average Error R-O, which is high given the answer lengths. It is likely that the difficulties DAVE is facing with multi-tasks stem from the naïve concatenation of tasks. In future we will explore multi-tasks where the “sub-tasks” are related.

4.5 Discussion and Limitations

The results presented are promising. DAVE has shown clear ability to produce syntactically correct Verilog (in our tests, it rarely, if ever, produced outputs that could not compile—errors were almost always related to operator choice and/or variable names). DAVE is capable of producing code with complex relationships between inputs and outputs, and even with intermediate signals. In total, DAVE returned the correct answer in **94.8 %** of all validation tests.

That said, our work has limitations. Firstly, other than inferring clocks, we do not yet ask DAVE to create a signal that was not already named or otherwise described (e.g., we never provide code such as “Output ‘a’ nor ‘b’”, it is always “Output ‘a’ nor ‘b’ in ‘c’.”). Likewise, we never rely on any form of *creativity* in the generated results—our training data suggests that there is only one path forward to the implementation for a given task template. That is, our templates had a *many-to-one* relationship with the Verilog they described, despite there being different ways to express functionally identical Verilog. These are the focus of our ongoing studies.

DAVE inherits some *technical* limitations of GPT-2: The model can only generate outputs of up to 1024 tokens (i.e., words, symbols). As longer snippets of code can potentially run into this limit, we had to limit certain inputs—sequence generators were capped at no more than 4 elements, and our multi-tasks were prevented from using long-winded descriptive register templates.

5 CONCLUSIONS

This paper set out to explore the potential use of ML for translating natural language specifications into their corresponding Verilog HDL. We adopted the GPT-2 language model and fine-tuned it over a large number of English/Verilog Task/Result pairs to produce DAVE. We investigated DAVE’s performance over sets of English to Verilog Tasks based on familiar and unfamiliar templates. In general, DAVE’s performance exceeded our expectations and was able to produce Verilog in response to both simple, prescriptive prompts, as well show success in acquiring the advanced capabilities required to solve more descriptive settings. Our future work will investigate the use of larger GPT-2 models for DAVE, increasing the complexity and length of the tasks, and tuning DAVE for specific tasks such as security assertion generation from natural language collateral.

TRY DAVE !

[Click here⁹](https://colab.research.google.com/drive/1aDSMDWL5hieB3_Th9ZdddDMAKQ2DjWxW) for instructions to run DAVE freely within Google Colab.

⁹https://colab.research.google.com/drive/1aDSMDWL5hieB3_Th9ZdddDMAKQ2DjWxW

ACKNOWLEDGMENTS

H. Pearce is supported by the National Science Foundation grant CMMI-1932264. B. Tan and R. Karri are supported in part by the Office of Naval Research under Award Number # N00014-18-1-2058. This work was supported in part by NYU CCS.

REFERENCES

- [1] BENGIO, Y., DUCHARME, R., VINCENT, P., AND JAUVIN, C. A Neural Probabilistic Language Model. *Journal of Machine Learning Research*, 3 (2003), 1137–1155.
- [2] BLACK, P. E. Ratcliff-Obershelp pattern recognition—dictionary of algorithms and data structures, 2004.
- [3] DEVLIN, J., CHANG, M., LEE, K., AND TOUTANOVA, K. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR abs/1810.04805* (2018).
- [4] DRECHSLER, R., HARRIS, I. G., AND WILLE, R. Generating formal system models from natural language descriptions. In *IEEE Int. High Level Design Validation and Test Workshop (HLDVT)* (2012), pp. 164–165.
- [5] HARRIS, C. B., AND HARRIS, I. G. Glact: Learning formal grammars to translate natural language specifications into hardware assertions. In *Design, Automation Test in Europe Conf. Exhibition (DATE)* (2016), pp. 966–971.
- [6] HERN, A. New ai fake text generator may be too dangerous to release, say creators. *The Guardian* (2019).
- [7] KAHNG, A. B. Machine Learning Applications in Physical Design: Recent Results and Directions. In *Int. Symp. Physical Design (ISPD)* (2018), pp. 68–73.
- [8] LIU, P., QIU, X., AND HUANG, X. Recurrent neural network for text classification with multi-task learning. *CoRR abs/1605.05101* (2016).
- [9] MIHALCEA, R., LIU, H., AND LIEBERMAN, H. NLP (Natural Language Processing) for NLP (Natural Language Programming). In *Computational Linguistics and Intelligent Text Processing* (2006), A. Gelbukh, Ed., Springer Berlin Heidelberg, pp. 319–330.
- [10] PAN, S. J., AND YANG, Q. A Survey on Transfer Learning. *IEEE Transactions on Knowledge and Data Engineering* 22, 10 (Oct. 2010), 1345–1359.
- [11] RADFORD, A., NARASIMHAN, K., SALIMANS, T., AND SUTSKEVER, I. Improving Language Understanding by Generative Pre-Training, 2018.
- [12] RADFORD, A., WU, J., CHILD, R., LUAN, D., AMODEI, D., AND SUTSKEVER, I. Language models are unsupervised multitask learners, 2019.
- [13] REDDY, S., CHEN, D., AND MANNING, C. D. Coqa: A conversational question answering challenge. *Transactions of the Association for Computational Linguistics* 7 (2019), 249–266.
- [14] SERVADEI, L., ZENNARO, E., DEVARAJEGOWDA, K., MANZINGER, M., ECKER, W., AND WILLE, R. Accurate Cost Estimation of Memory Systems Inspired by Machine Learning for Computer Vision. In *Design, Automation Test in Europe Conf. Exhibition (DATE)* (Mar. 2019), pp. 1277–1280.
- [15] SUNDERMEYER, M., SCHLÜTER, R., AND NEY, H. Lstm neural networks for language modeling. In *Conf. Int. Speech Communication Assoc.* (2012).
- [16] TREUDE, C., ROBILARD, M. P., AND DAGENAIS, B. Extracting Development Tasks to Navigate Software Documentation. *IEEE Transactions on Software Engineering* 41, 6 (June 2015), 565–581.
- [17] VAHID, F. *Digital Design with RTL Design, VHDL, and Verilog*. John Wiley & Sons, Mar. 2010.
- [18] VASWANI, A., SHAZEER, N., PARMAR, N., USZKOREIT, J., JONES, L., GOMEZ, A. N., KAISER, L., AND POLOSUKHIN, I. Attention is All you Need. In *Advances in Neural Information Processing Systems* 30, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds. Curran Associates, Inc., 2017, pp. 5998–6008.
- [19] WOOLF, M. minimaxir/aitextgen: A robust Python tool for text-based AI training and generation using GPT-2.
- [20] YU, C., XIAO, H., AND DE MICHELI, G. Developing synthesis flows without human knowledge. In *Design Automation Conf. (DAC)* (2018).
- [21] ZHAO, J., AND HARRIS, I. G. Automatic Assertion Generation from Natural Language Specifications Using Subtree Analysis. In *Design, Automation Test in Europe Conf. Exhibition (DATE)* (Mar. 2019), pp. 598–601. ISSN: 1558-1101.